

Bản trình chiếu: Cây lệch trái và ứng dụng

Huang Yuanhe

2005

Nguồn gốc: Đặc điểm và ứng dụng của cây lệch trái: bản trình chiếu

IOI China candidate team papers / 2005 / Huang Yuanhe / Huang Yuanhe.ppt

1 Mạch trình bày

Bản trình chiếu đi cùng bài viết *Đặc điểm và ứng dụng của cây lệch trái*. Văn bản trích xuất được từ PowerPoint có các mốc chính: *Leftist Tree*, *Mergeable Heap*, *Heap Ordered*, *Leftist Property*, *External Node*, $\text{dist}(i)$, các thao tác MAKENULL, MERGE, INSERT, MIN, DELETEMIN, DELETE, DECREASE, cùng bài ví dụ với $n \leq 10^6$, $0 \leq a_i \leq 2 \cdot 10^9$, trung vị và độ phức tạp $\mathcal{O}(n \log n)$. Bản ghi chú này tóm lược nội dung slide theo bài viết đã dịch; phần mã gộp chỉ được diễn giải, không chép lại thành khuôn dài.

2 Tính chất

Cây lệch trái là một hiện thực của đồng có thể gộp. Đồng nhị phân rất tốt cho chèn và xóa phần tử nhỏ nhất, nhưng gộp hai đồng nhị phân thường tốn $\mathcal{O}(n)$. Nếu thuật toán cần liên tục hợp nhất các hàng đợi ưu tiên, thao tác gộp trở thành nút thắt; cây lệch trái được thiết kế để giải quyết đúng điểm này.

Mỗi nút có khóa, con trái, con phải và một giá trị khoảng cách dist . Nút ngoài là nút có ít nhất một con rỗng. Khoảng cách của nút i , ký hiệu $\text{dist}(i)$, là số cạnh trên đường ngắn nhất từ i tới một nút ngoài trong cây con của nó; nút rỗng được quy ước có khoảng cách -1 .

Cây lệch trái thỏa hai tính chất. Thứ nhất là tính chất đồng: khóa của cha không lớn hơn khóa của con, nên gốc chứa phần tử nhỏ nhất. Thứ hai là tính chất lệch trái:

$$\text{dist}(\text{left}(x)) \geq \text{dist}(\text{right}(x)),$$

nên đường đi theo con phải luôn ngắn. Từ định nghĩa này suy ra

$$\text{dist}(x) = \text{dist}(\text{right}(x)) + 1.$$

Nếu một cây lệch trái có N nút, khoảng cách của gốc không vượt quá

$$\lfloor \log(N + 1) \rfloor - 1.$$

Đây là lý do mọi thao tác chỉ đi dọc nhánh phải có độ phức tạp logarit.

3 Thao tác gộp

Thao tác trung tâm của toàn bộ cấu trúc là

$$C \leftarrow \text{Merge}(A, B).$$

Nếu một cây rỗng, kết quả là cây còn lại. Nếu cả hai cây không rỗng, chọn gốc có khóa nhỏ hơn làm gốc mới; cây kia được gộp đệ quy vào con phải của gốc ấy. Sau khi gộp xong, có thể khoảng cách con phải lớn hơn con trái, làm mất tính chất lệch trái. Khi đó chỉ cần hoán đổi hai con, rồi cập nhật dist từ con phải mới.

Slide có mã giả MERGE, nhưng ý tưởng chỉ gồm ba bước trên: chọn gốc nhỏ hơn, gộp vào nhánh phải, rồi sửa lại bằng hoán đổi nếu cần. Vì mỗi bước đệ quy đi xuống một nhánh phải, số bước bị chặn bởi tổng khoảng cách của hai cây. Do đó

$$\text{Merge}(A, B) = \mathcal{O}(\log N_1 + \log N_2),$$

thường viết gọn là $\mathcal{O}(\log N)$.

Các thao tác khác đều quy về gộp. Chèn một phần tử về gộp cây hiện tại với cây một nút. Lấy phần tử nhỏ nhất chỉ cần đọc gốc. Xóa phần tử nhỏ nhất là xóa gốc rồi gộp hai cây con. Xóa một nút đã biết hoặc giảm khóa cần thêm con trở cha và một bước sửa dọc đường lên, nhưng vẫn dựa trên thao tác gộp và giữ được độ phức tạp $\mathcal{O}(\log N)$. Slide liệt kê các tên thao tác này để nhấn mạnh rằng cây lệch trái không chỉ là một mẹo gộp, mà là một hàng đợi ưu tiên hoàn chỉnh khi cần hợp nhất động.

4 Xây dựng và so sánh

Nếu chèn từng nút một, xây cây từ n phần tử tốn $\mathcal{O}(n \log n)$. Bài viết còn nêu cách dùng một hàng đợi: ban đầu mỗi phần tử là một cây một nút, mỗi lần lấy hai cây đầu hàng đợi gộp lại rồi đưa về cuối. Lịch gộp theo từng cặp làm kích thước các cây gần nhau, và tổng chi phí là $\mathcal{O}(n)$. Đây là một chi tiết hữu ích khi cần khởi tạo lớn.

So với đồng nhị phân, cây lệch trái tốn thêm con trở và trường dist, nên không phải lựa chọn tốt nhất cho mọi hàng đợi ưu tiên. Điểm mạnh của nó là thao tác gộp nhanh và các nút có vị trí vật lý ổn định hơn, thuận tiện khi cần xóa một nút đã biết. Slide cũng nhắc đồng Fibonacci: nó có một số độ phức tạp lý thuyết tốt hơn, nhưng cài đặt dài và dễ sai hơn. Trong bối cảnh thi lập trình, cây lệch trái thường hấp dẫn vì cân bằng giữa hiệu năng và độ đơn giản.

5 Ứng dụng: dãy không giảm gần nhất

Ví dụ chính của slide là bài *Dãy số* của Baltic 2004. Cho dãy

$$a_1, a_2, \dots, a_n, \quad n \leq 10^6, \quad 0 \leq a_i \leq 2 \cdot 10^9,$$

cần tìm dãy không giảm

$$b_1 \leq b_2 \leq \dots \leq b_n$$

sao cho

$$|a_1 - b_1| + |a_2 - b_2| + \dots + |a_n - b_n|$$

nhỏ nhất.

Nếu một đoạn của nghiệm có cùng giá trị, giá trị tối ưu của đoạn đó là trung vị của các a_i trong đoạn. Vì nghiệm b phải không giảm, thuật toán duy trì dãy các đoạn, mỗi đoạn có một trung vị w . Khi thêm

a_{k+1} , ta tạo một đoạn mới. Nếu trung vị đoạn trước lớn hơn trung vị đoạn sau, hai đoạn vi phạm tính không giảm và phải được gộp. Sau khi gộp, trung vị mới lại có thể vi phạm với đoạn trước nữa, nên tiếp tục gộp ngược cho tới khi các trung vị tăng dần.

Cấu trúc dữ liệu cần hỗ trợ hai việc: gộp tập phần tử của hai đoạn và biết trung vị của tập sau khi gộp. Ta dùng một đồng lớn cho mỗi đoạn, giữ nửa nhỏ hơn của các phần tử; đỉnh đồng là trung vị theo quy ước của bài viết. Khi hai đoạn gộp lại, chỉ cần gộp hai đồng, rồi xóa bớt đỉnh nếu số phần tử trong đồng vượt quá nửa kích thước đoạn. Vì các đồng phải gộp nhiều lần, cây lệch trái là lựa chọn tự nhiên. Tổng số lần gộp và xóa bị chặn bởi số phần tử, nên độ phức tạp toàn bài là

$$\mathcal{O}(n \log n).$$

6 Tổng kết

Thông điệp của bản trình chiếu là: cây lệch trái không phải cây cân bằng để tìm kiếm theo khóa, mà là đồng có thể gộp. Nó giữ đường phải ngắn bằng giá trị dist, nhờ vậy thao tác gộp nhanh; từ gộp suy ra chèn, xóa nhỏ nhất và các thao tác mở rộng. Trong các bài mà các tập ưu tiên liên tục hợp nhất, đặc biệt khi cần trung vị hoặc cực trị của các khối đang gộp, cây lệch trái thường cho lời giải vừa gọn vừa đủ nhanh.